VIII - Eliminación de superficies ocultas

Computación gráfica y ambientes virtuales (0494)

Facultad de Ingeniería
Centro Regional Universitario Córdoba - IUA
Universidad de la Defensa Nacional

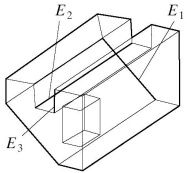
2026

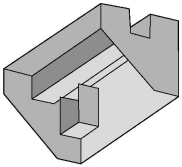


Superficies visibles

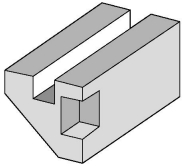
Introducción

- La mayoría de los objetos en una escena son opacos
- Una primer propuesta implica dibujar todas las caras de un objeto
- Sin embargo, sólo la cara que es visible al ojo debe ser realmente pintada
- ¿Cómo determinar cuál es la geometría correcta con la que se debe plasmar un objeto?





¿Rep. correcta?



¿Rep. correcta?



¿Rep. correcta?

Superficies visibles

Planteo del problema

Dado un conjunto de objetos 3D (escena) y una especificación de visualización (cámara) hay que determinar qué líneas o superficies de los objetos son visibles.

¿Porqué es necesario determinar qué objetos *no* son visibles?

- Determinar la apariencia correcta de los objetos.
- No se desea dibujar elementos que se encuentran fuera del área de pintado.
- No se desea perder tiempo.
- Evitar accesos a memoria innecesarios.

Superficies visibles

Estrategias para determinar la visibilidad

Recorte

- Remueve elementos no deseados dentro del área de pintado
- Se puede aplicar a una escena ó a objetos individuales (polígonos)

Oclusión

- Intenta identificar qué objetos, superficies o partes de superficies son visibles desde un ángulo de visión particular
- Maneja un conjunto diverso de primitivas geométricas

La *oclusión* también se conoce como:

- **Eliminación de Superficies Ocultas** (Hidden Surface Elimination)
- **Determinación de Superficies Visibles** (Visible Surface Determination)

Métodos de recortes

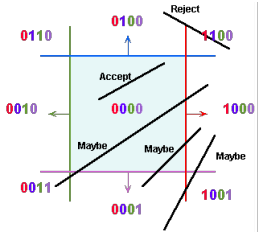
Recorte de línea básico

Estrategia: recortar líneas según los lados de la región de recorte

- Si ambos extremos están fuera de la región, descartar y parar
- Si ambos están dentro de la región, aceptar y proseguir
- Si un extremo esta dentro y el otro fuera, recortar el extremo en el punto de cruce y proseguir

Multiples implementaciones:

- Cohen-Sutherland
- Liang-Barsky
- Sutherland-Hodgman
- Weiler-Atherton

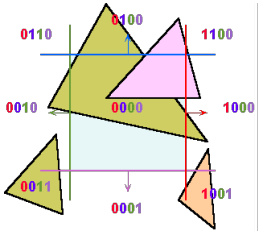


Métodos de recortes

Recorte de polígonos

Estrategia: Se extienden las mismas ideas que para el recorte de líneas

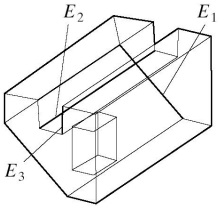
- Funcionan muy bien en 2D
- En los casos de intersecciones hay que generar nuevos bordes en los polígonos a partir de puntos de intersección
- Funcionan muy bien en 3D para objetos que no son complejos (ej. primitivas)
- En los casos de intersecciones hay que generar nuevos superficies en los objetos a partir de los segmentos de intersección



Métodos de oclusión

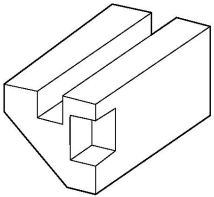
Introducción

- Métodos se originan para la visualización de mallas de alambre (wireframe)
- Se busca obtener una representación volumétrica a partir de la representación de líneas
- Problema resuelto en los métodos de representación clásicos (dibujo artístico / técnico)



Estrategia: remoción de líneas ocultas

- Las líneas por sí mismas no ocultan líneas
- Las líneas son bordes de superficies opacas que ocultan líneas
- Algunas técnicas muestran a las líneas como segmentos punteados



Determinación de Superficies Visibles

Dos estrategias generales

Estrategias basadas en imagen

- Aprovechan el resultado de la última etapa de proyección 2D (vol. vis. canónico)
- Cada pixel a muestrear de la imagen 2D tendrá asociado una línea de proyección
- Determinar cual de los n objetos es visible en *cada pixel* de la imagen

```
for (cada pixel en la imagen) {
   - determinar el objeto mas cercano al
     observador que es tocado por un proyector a
     través del pixel;
   - dibujar el pixel con el color apropiado;
}
```

Estrategias basadas en objetos

- Comparar *los objetos* entre sí, eliminando objetos enteros o porciones de ellos que no son visibles

```
for (cada objecto en el mundo) {
   - determinar aquellas partes del objeto cuya
     visión es obstruida por otras partes
     propias o de otros objetos;
   - dibujar las partes visibles con el color
     apropiado;
}
```

Determinación de Superficies Visibles

Tres tipos de algoritmos

Conservadores: realizan un rechazo trivial de las superficies

- Pueden ser "destructivos"
- No brinda una respuesta final!
- Utilizados para obtener resultados previos a otros algoritmos
- Algoritmos
  - Recorte por volumen de visualización canónico
  - Volúmenes envolventes (bounding volumes)
  - Recorte de cara trasera

Determinación de Superficies Visibles

Tres tipos de algoritmos (cont.)

En espacio de imagen: resuelven la visibilidad por medio de los puntos discretos de una imagen

- Son de precisión: determinan la visibilidad final
- Muestrean el modelo, **luego** resuelven la visibilidad
- Resuelven que objetos tiene sentido evaluar
- Algoritmos
  - Z-buffer
  - Buffer de profundidad de escaneo
  - Algoritmo del pintor
  - Algoritmos basados en raytracing

Determinación de Superficies Visibles

Tres tipos de algoritmos (cont.)

En el espacio de objeto: resuelven todas las posibles direcciones de visualización para un único punto de vista

- Son de precisión: determinan la visibilidad final
- Resuelven la visibilidad, **luego** muestrean lo resultante
- Independientes de la dirección de visualización o la densidad de muestreo
- Algoritmos
  - Particionado binario del espacio (Árboles BSP)
  - Recorte de polígonos
  - Ordenamiento por profundidad 3D

Consideración final

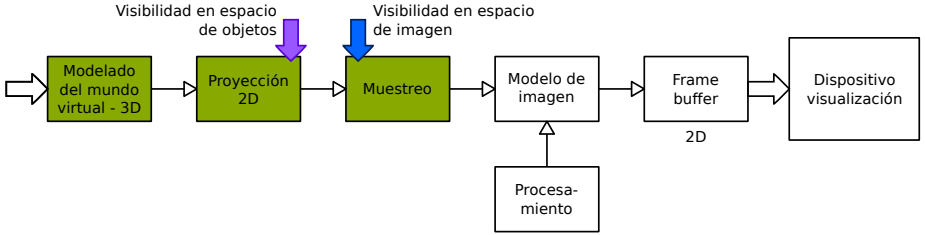
En el pipeline, usualmente, se utiliza primero algún algoritmo conservador y luego uno de precisión ya sea en el *espacio de imagen* o en el *espacio de objetos*.

Determinación de Superficies Visibles

Visibilidad y pipeline de rendering

Conjunto de procesos que permiten visualizar un modelo 3d abstracto por medio de una representación 2d.

- Transformación del modelo matemático del mundo a pixeles en la pantalla
- Este proceso requiere múltiples pasos que se realizan en secuencia
- La información salida de una etapa sirve como información de entrada de la siguiente



Determinación de Superficies Visibles

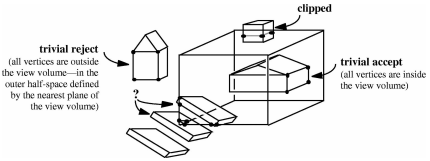
Consideraciones uso / selección de los algoritmos

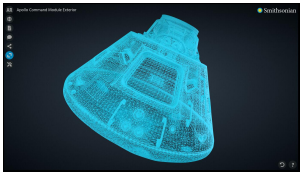
- cuál es la geometría a evaluar? triángulos vs. superficies implícitas
- se debe pre-procesar el modelo?
- qué porcentaje del escenario debe evaluarse?
- hay procesamiento de anti-aliasing?
- existen objetos transparentes?
- el algoritmo considera objetos que se desplazan?

Algoritmos conservadores

1. Recorte por volumen de visualizavión

- Los poliedros transformados en el mundo normalizado, luego son recortados según los bordes del volumen de visualización canónico; un polígono por vez
- Los polígonos son recortados de a un borde por vez
- Los cálculos de intersección son triviales debido a la formulación del volumen de visualización
- Es necesario crear nuevos vértices cuando los objetos son recortados
- Sería conveniente poder rechazar grupos de objetos rápidamente





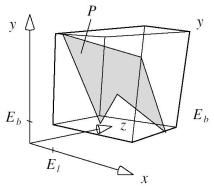
¿Como puedo acelerar la evaluación de un modelo poligonal complejo?

Algoritmos conservadores

2. Volúmenes envolventes (Bounding volumes - BV)

Estrategia: intentar determinar intersecciones de objetos de geometría compleja usando una primitiva simple que acelere los procesos

- Delimitar cada objeto complejo con un objeto simple (primitivas)
  - esfera
  - cubo
  - poliedro
- Si el volumen envolvente no es visible, tampoco lo es el objeto en su interior
- Un volumen envolvente efectivo depende de la orientación y forma del objeto
- Se pueden colocar varios objetos en un único volumen: aumenta la eficiencia



Algoritmos conservadores

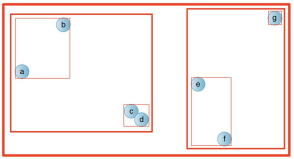
2. Volúmenes envolventes (Bounding volumes - BV)

Aplicación de BV: jerarquía de objetos en una escena (Bounding Volumes Hierarchy - BVH)

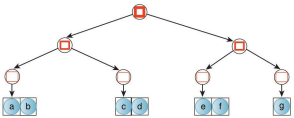
Permite organizar una escena para poder determinar que objetos deben ser recortados.

Dos métodos de construcción:

1. En forma repetitiva agrupar volúmenes de objetos cercanos hasta que la escena completa se encuentre delimitada
   - requiere de alguna forma de ordenamiento espacial
2. Crear un grafo de la escena
   - Volumen envolvente en un nodo es igual a la unión de los volúmenes envolventes de los hijos
   - Es fácil de crear



Estructura espacial (2D)

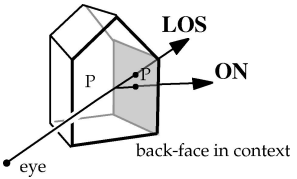


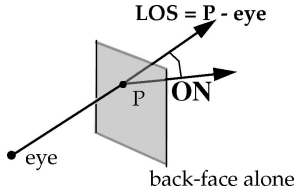
Estructura lógica

Algoritmos conservadores

3. Recorte de cara trasera (Back face culling)

- El enfoque considera los objetos como poliedros cerrados, con el ojo observando siempre desde afuera de ellos
- La línea de visión (Line of Sight (LOS)), es el proyector que emana del centro de proyección (center of projection (COP)) hacia cualquier punto P del polígono.
- En proyecciones paralelas LOS = DOP (dirección de la proyección)
- Para el test, utiliza la normal de los polígonos (planos) de cada objeto (outward normal (ON)) en sentido dextrógiro
- Si la normal se orienta en la misma dirección del LOS, entonces la cara es trasera
  - if $LOS \cdot ON > 0$, then el polígono es invisible - descartar
  - if $LOS \cdot ON < 0$, then el polígono puede ser visible





Algoritmos de precisión en espacio de imagen

Introducción

Idea básica

- Encontrar el punto visible más cercano a través de cada pixel de la imagen
- Los cálculos se realizan acorde con la resolución de imagen (ej. m x m)
- No requiere gran precisión

Complejidad

- Enfoque Naïve que chequea todos los n objetos en cada pixel. $O(n, m^2)$
- Mejores enfoques solo chequean los objetos que podrían ser visibles en cada pixel
- Si en promedio d objetos son visibles en cada pixel, luego $O(d, m^2)$

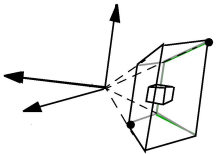
Implementación

- Muy simple de implementar

Algoritmos de precisión en espacio de imagen

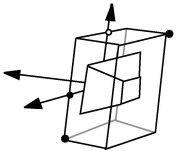
1. Algoritmo Z-buffer: revisión de transformación en perspectiva

Antes de la transformación

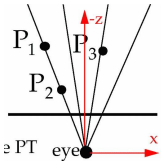


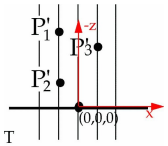
Volumen de visualización canónico en perspectiva de un cubo

Después de la transformación



Volumen de visualización paralelo normalizado y cubo distorsionado





Algoritmos de precisión en espacio de imagen

1. Algoritmo Z-buffer: revisión de transformación en perspectiva

Recorte

- El recorte luego de la transformación es simple utilizando los planos definidos
  - $-1 < x < 1$, $-1 < y < 1$, $0 < z < -1$

Comparación de profundidad

- Si se realiza *antes* de la transformación
  - Comparar de a pares de puntos P_1 y P_2 para determinar si están en el mismo proyector
  - Si el par (x, y) de cada punto están en el mismo proyector, luego se evalúan los valores de z
- Si se realiza *después* de la transformación
  - Comparar de a pares de puntos P'_1 y P'_2 para determinar si están en el mismo proyector
  - Se debe cumplir que $x'_1 = x'_2$ e $y'_1 = y'_2$ (muy sencillo!)
  - Si los pares (x', y') de los puntos son iguales, luego se evalúan los valores de z

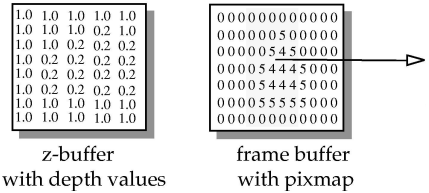
Conclusión: luego de la transformación los puntos simplemente se descartan por valores de z!

Algoritmos de precisión en espacio de imagen

1. Algoritmo Z-buffer

Estrategia para evaluar la visualización

- Sumar un *buffer* adicional además del *framebuffer* para almacenar los valores de profundidad z de los pixeles
- Esto permitirá realizar la determinación de las superficies visibles



Estructura de buffers a utilizar

- *Buffer de intensidades*
  - es el buffer de color de pixels RGB
  - se inicializa con un color de fondo
- *Buffer de profundidad ("Z")*
  - mantiene la profundidad de la escena en cada pixel
  - está inicializado a la máxima profundidad

Algoritmos de precisión en espacio de imagen

1. Algoritmo Z-buffer

- Se dibuja cada objeto/polígono que no se puede rechazar trivialmente
- Los polígonos son digitalizados en cualquier orden
- Si se encuentra una pieza que debe ser pintada (uno o más pixels), se pinta por encima de lo que existía en el buffer
- Cuando los pixels se superponen, utilizar el Z-buffer para decidir que polígono se pinta en el pixel

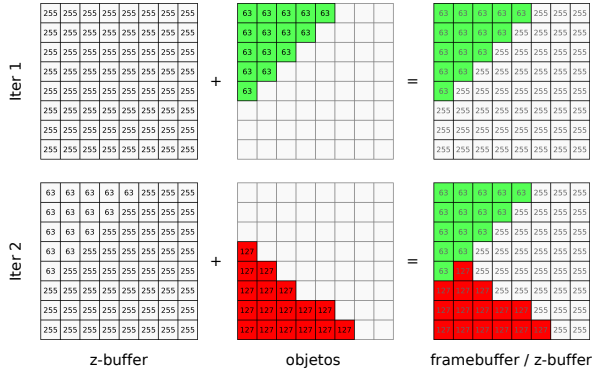
```
for each pixel (i,j) do
    Z-buffer[i,j] ← FAR
    Framebuffer[i,j] ← <background color>
end for
for each polygon A do
    for each pixel in A do
        Compute depth z and shade s of A at (i,j)
        if z < Z-buffer[i,j] then
            Z-buffer[i,j] ← z
            Framebuffer[i,j] ← s
        end if
    end for
end for
```

Pseudo-código del algoritmo z-buffer

Algoritmos de precisión en espacio de imagen

1. Algoritmo Z-buffer

Ejemplo utilizando un Z-buffer de enteros con near = 0, far = 255



Algoritmos de precisión en espacio de imagen

1. Algoritmo Z-buffer: ventajas

- Simplicidad – permite su implementación en hardware!
  - actualmente utilizado por todas las tarjetas gráficas
- Los polígonos pueden evaluarse en cualquier orden. Paralelismo!!
- Sólo se considera a los polígonos
  - ...la oclusión sigue siendo un problema global!
  - Fuerza bruta, pero rápido!
- El Z-buffer puede almacenarse junto con la imagen, permitiendo componer imágenes, en lugar de modificar escenarios (animación)
  - gran beneficio para escenarios complejos
  - todos los algoritmos de VSD pueden generar un Z-buffer
- Puede utilizarse con superficies no poligonales: ej. CSGs (intersect, union, difference), $z = f(x, y)$.
- Se requiere procesamiento y memoria, aunque la imagen puede ser dividida en su procesamiento..!!

Algoritmos de precisión en espacio de imagen

1. Algoritmo Z-buffer: desventajas

- Problema de precisión: recorte por proyección en perspectiva
  - la compresión del eje z producto de la transformación en perspectiva
  - los objetos que originalmente se encuentran lejos terminan con valores de z cercanos y pequeños
  - se pierde precisión en la información de profundidad lo cual genera errores en el plasmado por objetos pequeños a la distancia
  - los polígonos co-planares muestran una "pelea de z"
  - el uso de flotantes reduce el problema... no lo elimina
- No se puede realizar anti-aliasing
  - requiere conocer todos los polígonos involucrados en un pixel (vecindario de polígonos)
  - concepto de A-buffer para el tratamiento de anti-aliasing
- Algoritmo original evalua todos los objetos (aún no recortados)
  - Z-buffering jerárquico (Greene & Kass SIGGRAPH '94)

Algoritmos de precisión en espacio de objetos

Introducción

Idea básica

- Opera sobre los objetos en sí mismos: Los objetos serán tratados como "primitivas"
- Los objetos comúnmente se interponen unos con otros
- El test se realiza a alto nivel
- La lista de objetos visibles puede luego ser plasmada en diferentes resoluciones

Complejidad

- Para n objetos computar la visibilidad puede llevar $O(n^2)$
- Para una pantalla (imagen) de $m \times m$ pixels, se debe evaluar m^2 pixels
- La complejidad total sería $O(k_{obj} \cdot n^2 + k_{disp} \cdot m^2)$

Implementación

- Generalmente complejos

Algoritmos de precisión en espacio de objetos

Primeras propuestas

Remoción de Líneas Ocultas - Roberts '63

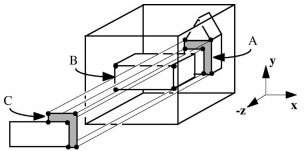
- Comparar cada lado con cada objeto, eliminando los lados invisibles o partes de lados
- Los objetos deben ser poliedros convexos

Complejidad

- Peor a $O(n^2)$ dado que cada objeto debe ser comparado con todos los lados

Enfoque similar para superficies ocultas

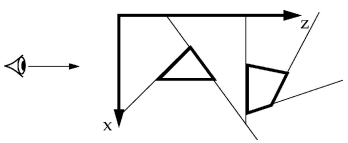
- cada polígono es recortado usando las proyecciones de los otros polígonos por delante de él
- se eliminan las superficies invisibles y se crean sub-polígonos visibles
- lento, sin mucha precisión en casos particulares

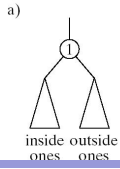


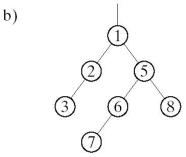
Algoritmos de precisión en espacio de objetos

1. Particionado binario del espacio - (Binary Space-Partitioning - BSP)

- Provee una subdivision espacial y orden de pintado
- Estrategia: Divide y conquista
  - divide recursivamente el espacio en una jerarquía de semi-espacios, mediante la división de polígonos en esos semi-espacios, usando el plano del polígono de referencia
  - construye un árbol BSP que representa la jerarquía
  - cada polígono seleccionado es la raíz de un sub-árbol
  - permite realizar pre-cálculos para procesamientos independientes de la dirección de visualización (animaciones)



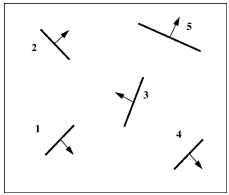




Algoritmos de precisión en espacio de objetos

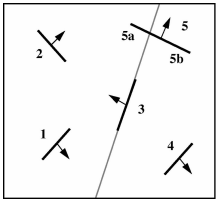
1. Particionado binario del espacio - (Binary Space-Partitioning - BSP)

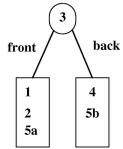
- Para mostrar cualquier polígono correctamente
  - plasmar primero todos los polígonos que se encuentren por detrás del polígono actual (en relación con el punto de vista), luego el polígono actual, luego todos los polígonos por delante.
  - cómo determinar el orden?. Elegir un polígono y procesar en forma recursiva



Paso 1:

- Elegir cualquier polígono (ej. polígono 3)
- Subdividir el espacio con los otros polígonos a partir de este plano
- Subdividir polígonos cuando sea necesario.



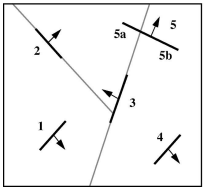


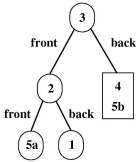
Algoritmos de precisión en espacio de objetos

1. Particionado binario del espacio - (Binary Space-Partitioning - BSP)

Paso 2:

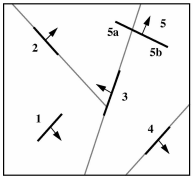
- Procesar el sub-árbol *frontal* recursivamente

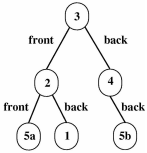




Paso 3:

- Procesar el sub-árbol *trasero* recursivamente





Algoritmos de precisión en espacio de objetos

1. Particionado binario del espacio - (Binary Space-Partitioning - BSP)

Realizar el árbol BSP dependiente de la vista cada vez que el punto de vista cambia:

- Para pintar primero se recorren la rama *trasera* del nodo y luego se pinta la rama *frontal*
- Los nodos pueden visitarse en orden back-to-front utilizando un recorrido modificado in-order del árbol
- En cada nodo, el lado negativo son los polígonos que están en el lado en que no se encuentra el punto de vista del nodo (lado no visible del nodo)
- Muy rápido! Quake III utilizaba este método para acelerar su evaluación de intersecciones
- BSP se crea en la etapa de *proyección 2D* (coordenadas del mundo) una vez que tengo los datos de visualización

Algoritmos de precisión en espacio de objetos

1. Particionado binario del espacio - (Binary Space-Partitioning - BSP)

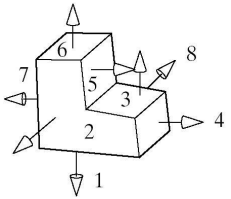
Ejemplo

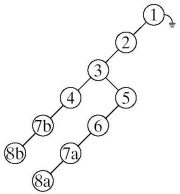
1. Crear el árbol BSP.

procedure *MakeBSPTree*:
takes *PolygonList L*
returns *BSPTree*

Choose polygon *A* from *L* to serve as root
Split all polygons in *L* according to *A*
node ← *A*
node.neg ← *MakeBSPTree*(polygons on neg. side of *A*)
node.pos ← *MakeBSPTree*(polygons on pos. side of *A*)
return *node*

end procedure





Resultado de la construcción

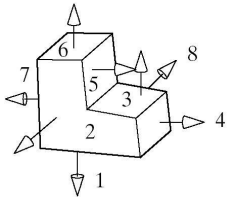
Algoritmos de precisión en espacio de objetos

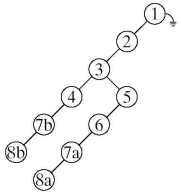
1. Particionado binario del espacio - (Binary Space-Partitioning - BSP)

Ejemplo

2. Pintar los polígonos.

```
procedure DisplayBSPTree:
Takes BSPTree T
    if T is empty then return
    if viewer is in front (on pos. side) of T.node then
        DisplayBSPTree(T.neg)
        Draw T.node
        DisplayBSPTree(T.pos)
    else
        DisplayBSPTree(T.pos)
        Draw T.node
        DisplayBSPTree(T.neg)
    end if
end procedure
```





Árbol a utilizar para pintar

Resultado de pintado:

1 – 7b – 8b – 4 – 3 – 7a – 8a – 6 – 5 - 2

Bibliografía recomendada

- HEARN, D. & BAKER, M. P.. *Computer Graphics with OpenGL*. 4ta ed. Essex (UK): Pearson Education Limited, 2014. cap. 14 Visible-surface detections methods, p. 465-493.

- HUGHES J. F., FOLEY, J. D., VAN DAM, A., FEINER, S. K.. *Computer Graphics, Principles and Practice*. 3ra ed. New Jersey (USA): Addison Wesley Publishing Company, 2014. cap. 36 Visibility determination, p. 1023-1063.

¿¿¿ PREGUNTAS ???